Cheatsheet ICPC — Python + C++

Hoja de referencia · probado con g++ -std=c++20 y Python 3

0. Índice y cómo usar esta hoja

Busca por el código entre corchetes: cada entrada tiene el mismo formato — **cuándo usarla**, **código**, **trampa**. Todo el código C++ y Python de esta hoja fue compilado/ejecutado y probado contra fuerza bruta.

Código	Sección	Úsala cuando…
[Q]	Búsqueda rápida "quiero hacer X"	Sabes QUÉ hacer pero no la sintaxis
[P]	Python por tema	Strings, simulación, números grandes, N pequeño
[C]	C++ por tema (STL)	Todo lo demás
[A]	Básicos: prefijos, búsqueda binaria, dos punteros	Rangos, "mínimo X tal que…", ventanas
[G]	Grafos y árboles	Nodos/aristas, caminos, árboles, matching, flujo
[D]	Estructuras de datos	Muchas consultas/actualizaciones en rango
[S]	Strings avanzados	Comparar subcadenas, buscar patrones, paréntesis
[M]	Matemática	mod 1e9+7, primos, divisores, bases, juegos
[GE]	Geometría en enteros	Puntos, polígonos, envolvente convexa
[T]	Esqueletos completos	Empezar un problema desde cero
[R]	Mapa regional	Reconocer el patrón de un problema
[J]	Checklist del juez	Antes de CADA envío

Regla de lenguaje (decídelo al leer las cotas):

Situación	Lenguaje
Strings/simulación con N ≤ 10^4, aritmética con enteros gigantes, parsing	Python
N ≥ 10^5, grafos, árboles, DP grandes, cualquier estructura de [D]	C++
Dudas	C++ (los límites de tiempo se calibran con soluciones compiladas)

Operaciones por segundo aproximadas: C++ ≈ 10^8, Python ≈ 10^7 (o menos con bucles anidados).

N máximo	Complejidad que pasa
≤ 12	O(N!)
≤ 20–25	O(2^N · N)
≤ 500	O(N^3)
≤ 5 000	O(N^2)
≤ 10^6	O(N log N)
≤ 10^8	O(N)
> 10^9	O(√N) o O(log N)

[Q] Búsqueda rápida: "quiero hacer X"

Busca la intención en la columna izquierda. En C++ se asume `#include <bits/stdc++.h>`, `using namespace std;` y `typedef long long ll;` (ver [C1]).

Q1 — Leer entrada

Quiero…	Python	C++
Un entero	`n = int(input())`	`int n; cin >> n;`
Dos enteros en una línea	`x, y = map(int, input().split())`	`cin >> x >> y;`
Una línea de N enteros → lista	`a = list(map(int, input().split()))`	`vector<ll> a(n); for (auto &x : a) cin >> x;`
Una palabra / string sin espacios	`s = input().strip()`	`string s; cin >> s;`
Una línea completa con espacios	`line = input()`	`getline(cin, line);` (si antes usaste `cin >>`, llama `getline` una vez extra para comer el salto)
Matriz R×C de números	`m = [list(map(int, input().split())) for _ in range(R)]`	`vector<vector<int>> m(R, vector<int>(C)); for (auto &f : m) for (auto &x : f) cin >> x;`
Grilla de caracteres	`g = [input().strip() for _ in range(R)]`	`vector<string> g(R); for (auto &f : g) cin >> f;`
Todo lo que queda (hasta EOF)	`rest = sys.stdin.read().split()`	`while (cin >> x) { ... }`
Leer rápido (N ≥ 10^5)	`import sys; input = sys.stdin.readline` (luego usa `.strip()` en strings)	`ios::sync_with_stdio(false); cin.tie(nullptr);`

Q2 — Imprimir

Quiero…	Python	C++
Lista separada por espacios	`print(*a)`	`for (int i = 0; i < n; i++) cout << a[i] << " \n"[i == n - 1];`
Un valor por línea	`print(*a, sep="\n")`	`for (auto x : a) cout << x << "\n";`
Decimal con 2 cifras	`print(f"{x:.2f}")`	`cout << fixed << setprecision(2) << x << "\n";`
Muchas líneas rápido	`sys.stdout.write("\n".join(out) + "\n")`	usa `"\n"`, nunca `endl`
Sí/No	`print("YES" if ok else "NO")`	`cout << (ok ? "YES" : "NO") << "\n";`

Q3 — Strings

Quiero…	Python	C++
Contar ocurrencias de un carácter	`s.count("a")`	`count(s.begin(), s.end(), 'a')`
Posición de una subcadena (−1 si no está)	`s.find("na")`	`s.find("na")` (devuelve `string::npos` si no está)
¿Contiene?	`"na" in s`	`s.find("na") != string::npos`
Subcadena desde i, largo k	`s[i:i+k]`	`s.substr(i, k)`
Invertir	`s[::-1]`	`reverse(s.begin(), s.end());`
Reemplazar	`s.replace("a", "o")`	(a mano con un bucle)
Dividir por espacios / por coma	`s.split()` / `s.split(",")`	`stringstream ss(line); while (ss >> w) ...`
Unir lista de strings	`" ".join(lst)`	bucle con +=
String → número	`int(s)`	`stoi(s)` / `stoll(s)`
Número → string	`str(x)`	`to_string(x)`
Dígito carácter → valor	`int(c)` u `ord(c) - 48`	`c - '0'`
Letra → índice 0..25	`ord(c) - ord('a')`	`c - 'a'`
Mayúsculas / minúsculas	`s.upper()` / `s.lower()`	`toupper(c)` / `tolower(c)`
¿Es dígito / letra?	`c.isdigit()` / `c.isalpha()`	`isdigit(c)` / `isalpha(c)`
Frecuencia de cada letra	`Counter(s)`	`int f[26] = {}; for (char c : s) f[c - 'a']++;`
Caracteres ordenados	`"".join(sorted(s))`	`sort(s.begin(), s.end());`

Q4 — Listas / vectores

Quiero…	Python	C++
Contar cuántas veces aparece x	`a.count(x)`	`count(a.begin(), a.end(), x)`
Máximo / mínimo / suma	`max(a)`, `min(a)`, `sum(a)`	`*max_element(a.begin(), a.end())`, `accumulate(a.begin(), a.end(), 0LL)`
Índice del máximo	`a.index(max(a))`	`max_element(a.begin(), a.end()) - a.begin()`
Ordenar ascendente / descendente	`a.sort()` / `a.sort(reverse=True)`	`sort(a.begin(), a.end());` / `sort(a.rbegin(), a.rend());`
Ordenar por 2 criterios (1.º desc, 2.º asc)	`a.sort(key=lambda p: (-p[0], p[1]))`	`sort(..., [](auto &x, auto &y){ if (x.first != y.first) return x.first > y.first; return x.second < y.second; });`
Quitar duplicados	`sorted(set(a))`	`sort(...); a.erase(unique(a.begin(), a.end()), a.end());`
Último elemento	`a[-1]`	`a.back()`
Recorrer con índice	`for i, v in enumerate(a):`	`for (int i = 0; i < n; i++)`
Recorrer dos listas a la vez	`for u, v in zip(a, b):`	`for (int i = 0; i < n; i++) { a[i]; b[i]; }`
Primer índice con a[i] ≥ x (lista ordenada)	`bisect_left(a, x)`	`lower_bound(a.begin(), a.end(), x) - a.begin()`
Primer índice con a[i] > x	`bisect_right(a, x)`	`upper_bound(a.begin(), a.end(), x) - a.begin()`
Lista de N ceros / matriz	`[0]*n` / `[[0]*C for _ in range(R)]`	`vector<ll> v(n, 0);` / `vector<vector<int>> m(R, vector<int>(C, 0));`
¿Alguno / todos cumplen?	`any(x > 7 for x in a)` / `all(...)`	`any_of(a.begin(), a.end(), [](ll x){ return x > 7; })`

Q5 — Contar, agrupar, buscar rápido

Quiero…	Python	C++
Diccionario contador	`d = defaultdict(int); d[k] += 1`	`map<string,int> d; d[k]++;`
Agrupar valores por clave	`d = defaultdict(list); d[k].append(v)`	`map<int, vector<int>> d; d[k].push_back(v);`
Lo más frecuente	`Counter(a).most_common(1)`	recorrer el map buscando el mayor
¿Ya vi este valor?	`vis = set(); x in vis`	`set<int> vis; vis.count(x)`
Sacar siempre el menor	`heapq.heappush(h, x); heapq.heappop(h)`	`priority_queue<ll, vector<ll>, greater<ll>> pq;`
Sacar siempre el mayor	push `-x`, pop y niega	`priority_queue<ll> pq;`
Cola por ambos lados	deque: `append`, `appendleft`, `pop`, `popleft`	deque: `push_back`, `push_front`, `pop_back`, `pop_front`

Q6 — Números

Quiero…	Python	C++
División entera / resto	`a // b`, `a % b`	`a / b`, `a % b` (¡con negativos difiere! ver [P7])
Redondear hacia arriba a/b (positivos)	`(a + b - 1) // b`	`(a + b - 1) / b`
MCD / MCM	`math.gcd(a, b)`, `math.lcm(a, b)`	`gcd(a, b)`, `lcm(a, b)`
Raíz entera	`math.isqrt(n)`	`ll r = sqrtl(n); while (r*r > n) r--; while ((r+1)*(r+1) <= n) r++;`
Potencia mod m	`pow(b, e, m)`	`pw(b, e, m)` en [M1]
Combinaciones n en k (exacto)	`math.comb(n, k)`	`C(n, k)` mod p en [M1]
Binario	`bin(x)[2:]`, `int("1010", 2)`	`bitset<32>(x).to_string()`, `stoll("1010", nullptr, 2)`
Contar bits en 1	`x.bit_count()` (Python ≥ 3.10) o `bin(x).count("1")`	`__builtin_popcountll(x)`
Infinito	`float('inf')` o `10**18`	`const ll INF = 4e18;`

[P] Python por tema

[P1] Plantilla base — Cópiala al empezar cualquier problema en Python.

```
import sys
from collections import Counter, defaultdict, deque
import heapq, bisect, math, itertools
input = sys.stdin.readline          # lectura rapida
sys.setrecursionlimit(10**6)        # solo si usas recursion

def main():
    n = int(input())
    a = list(map(int, input().split()))
    s = input().strip()             # .strip() OBLIGATORIO con readline
    print(*a)

main()
```

Trampa: con `input = sys.stdin.readline` el string trae el `\n` al final; `int()` y `split()` lo ignoran, pero `s` sin `.strip()` no.

[P2] Ejemplo corregido — contar días en que aparece un nuevo máximo (el ejercicio de hoy).

```
n = int(input())
a = list(map(int, input().split()))   # toda la linea -> lista de enteros, una sola vez
mx, dias = a[0], 1
for v in a[1:]:
    if v > mx:
        mx = v
        dias += 1
print(dias)
```

[P3] Strings — son inmutables: para modificar, pasa a lista y luego `"".join(lista)`.

```
s = "banana"
s.count("a")      # 3
s.find("n")       # 2   (-1 si no esta)
s.rfind("n")      # 4   (ultima aparicion)
s.index("n")      # 2   (ValueError si no esta)
s[1:4]            # "ana"  (fin exclusivo)
s[::-1]           # "ananab"
s.replace("a","o")# "bonono"
s.startswith("ba") / s.endswith("na")
s.split() / s.split(",") / " ".join(lst)
s.strip() / s.upper() / s.lower() / s.isdigit() / s.isalpha()
"12".zfill(4)     # "0012"
ord("a") -> 97    chr(98) -> "b"
lista = list(s); lista[0] = "B"; s = "".join(lista)
```

[P4] Listas

```
a.append(x); a.pop(); a.pop(0)   # pop(0) es O(n): usa deque
a.count(x); a.index(x); a.reverse(); a.sort()
sorted(a, reverse=True)
sorted(pares, key=lambda p: (-p[0], p[1]))   # 1.o desc, 2.o asc
max(range(n), key=lambda i: a[i])            # indice del maximo
max(words, key=len)
[x*x for x in a if x % 2 == 0]               # comprension
m = [[0]*C for _ in range(R)]                # NUNCA [[0]*C]*R (filas compartidas)
pre = [0]
for v in a: pre.append(pre[-1] + v)          # suma a[l..r] = pre[r+1] - pre[l]
```

[P5] Diccionarios, sets y contadores

```
d = {}; d[k] = d.get(k, 0) + 1
d = defaultdict(int); d[k] += 1           # no falla si k no existe
g = defaultdict(list); g[u].append(v)     # lista de adyacencia
c = Counter(s); c["a"]; c.most_common(2)  # [("a",3),("n",2)]
for k, v in d.items(): ...
for k in sorted(d): ...                   # claves en orden
st = set(); st.add(x); st.discard(x); x in st   # O(1)
```

[P6] Estructuras

```
q = deque([s]); q.append(x); q.appendleft(x); q.popleft(); q.pop()
h = []; heapq.heappush(h, (dist, nodo)); d, u = heapq.heappop(h)   # min-heap
heapq.heappush(h, -x); mayor = -heapq.heappop(h)                   # max-heap
b = [1,3,3,7]
bisect.bisect_left(b, 3)   # 1  primer indice con b[i] >= 3
bisect.bisect_right(b, 3)  # 3  primer indice con b[i] > 3
bisect.insort(b, 5)        # insertar manteniendo orden (O(n))
```

[P7] Matemática e itertools

```
math.gcd(a,b); math.lcm(a,b); math.isqrt(n); math.comb(n,k); math.perm(n,k);
math.factorial(n)
pow(b, e, m)          # potencia modular
pow(a, -1, m)         # inverso modular (m primo o coprimo)
divmod(17, 5)         # (3, 2)
-7 // 2  -> -4        # Python redondea hacia -inf
-7 % 3   -> 2         # resto siempre >= 0 (en C++ da -1)
bin(10)[2:]; int("1010", 2); int("ff", 16); hex(255)
itertools.permutations(a, 2); itertools.combinations(a, 2)
itertools.product([0,1], repeat=n)   # todas las mascaras
itertools.accumulate(a)              # sumas prefijas
```

[P8] Trampas de Python en el juez

Trampa	Solución
TLE con N ≥ 10^5 y bucles anidados	Pasa a C++
Recursión profunda (DFS) revienta	`sys.setrecursionlimit` no basta: usa pila explícita o C++
`print` dentro de un bucle grande es lento	Junta en lista y haz un solo `sys.stdout.write`
`[[0]*C]*R` comparte filas	`[[0]*C for _ in range(R)]`
Comparar strings numéricos (`"10" < "9"` es True)	Convierte con `int()` antes
Nombre del archivo	Debe ser la letra del problema: `A.py3` (ver [J])

[C] C++ por tema (STL)

[C1] Plantilla base — Todas las plantillas de [A]–[GE] asumen este encabezado. Compila con `g++ -std=c++20 -O2 -Wall A.cpp -o A`.

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
const ll INF = 4e18;
const ll MD = 1e9 + 7;

int main(){
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int n; cin >> n;
    vector<ll> a(n);
    for (auto &x : a) cin >> x;
    // ... resolver ...
    cout << "respuesta" << "\n";
}
```

[C2] Strings

```
string s; cin >> s;                    // una palabra
string line; getline(cin, line);       // linea completa (tras cin >>, llama getline 2 veces)
s.size(); s[i]; s += 'x'; s += "abc"; s.push_back('x'); s.pop_back();
s.substr(i, k);                        // desde i, largo k
s.find("na");                          // string::npos si no esta
count(s.begin(), s.end(), 'a');
reverse(s.begin(), s.end()); sort(s.begin(), s.end());
stoi(s); stoll(s); to_string(x);
c - '0'; c - 'a'; (char)('a' + k); isdigit(c); isalpha(c); toupper(c);
stringstream ss(line); string w; while (ss >> w) palabras.push_back(w);
```

[C3] Vectores y ordenamiento

```
vector<int> v(n, 0); v.push_back(x); v.pop_back(); v.back(); v.size(); v.clear();
vector<vector<int>> m(R, vector<int>(C, 0));
sort(v.begin(), v.end());              // ascendente
sort(v.rbegin(), v.rend());            // descendente
sort(p.begin(), p.end(), [](auto &x, auto &y){
    if (x.first != y.first) return x.first > y.first;   // 1.o desc
    return x.second < y.second; });                    // 2.o asc
*max_element(v.begin(), v.end()); max_element(v.begin(), v.end()) - v.begin();
accumulate(v.begin(), v.end(), 0LL);   // 0LL, no 0, o se desborda
lower_bound(v.begin(), v.end(), x) - v.begin();   // primer >= x
upper_bound(v.begin(), v.end(), x) - v.begin();   // primer > x
v.erase(unique(v.begin(), v.end()), v.end());     // tras sort: quitar duplicados
next_permutation(v.begin(), v.end());             // do { ... } while (next_permutation(...));
pair<int,int> pr = {1, 2}; pr.first; pr.second;
for (auto &[k, val] : pares) { ... }              // desempaquetar
```

[C4] set, multiset, map

```
set<int> st; st.insert(x); st.erase(x); st.count(x);   // ordenado, sin repetidos, O(log n)
*st.begin(); *st.rbegin();                             // menor / mayor
*st.lower_bound(x);                                    // menor elemento >= x (verifica != st.end())
multiset<int> ms; ms.erase(ms.find(x));                // borra UNA copia (ms.erase(x) borra todas)
map<string,int> mp; mp[k]++; mp.count(k);              // ordenado por clave
for (auto &[k, v] : mp) { ... }
unordered_map<int,int> um;                             // O(1) promedio, sin orden
```

[C5] Colas, pilas, heaps

```
queue<int> q; q.push(x); q.front(); q.pop(); q.empty();
stack<int> sk; sk.push(x); sk.top(); sk.pop();
deque<int> dq; dq.push_front(x); dq.push_back(x); dq.front(); dq.back();
priority_queue<int> mx;                                  // saca el MAYOR
priority_queue<int, vector<int>, greater<int>> mn;       // saca el MENOR
priority_queue<pair<ll,int>, vector<pair<ll,int>>, greater<>> pq;   // Dijkstra
```

[C6] Bits y utilidades

```
__builtin_popcountll(x);      // bits en 1
__lg(x);                      // posicion del bit mas alto (floor log2)
x & (1 << i);                 // bit i encendido?
for (int mask = 0; mask < (1 << n); mask++)   // todos los subconjuntos
gcd(a, b); lcm(a, b);
bitset<32>(x).to_string();
cout << fixed << setprecision(6) << x;
min({a, b, c}); max({a, b, c}); swap(a, b);
```

[C7] Trampas de C++

Trampa	Solución
Desborde: `int` llega a ~2·10^9	Usa `ll` para sumas, productos y distancias; `1LL * a * b`
`endl` en bucles grandes	Usa `"\n"`
`cerr` de depuración en el envío	Bórralo: cuenta en el límite de salida (ver [J])
`-7 % 3 == -1`	`((x % m) + m) % m`
`v.size() - 1` con vector vacío	Es unsigned: da un número gigante. Usa `(int)v.size() - 1`
`accumulate(..., 0)`	`0LL`
Arreglo local enorme (`int a[1000000]` en `main`)	Decláralo global o usa `vector`
Comparar `double` con `==`	`fabs(a - b) < 1e-9`, o trabaja en enteros

[A] Básicos: prefijos, búsqueda binaria, dos punteros

[A1] Sumas prefijas 1D — Cuándo: muchas consultas "suma de a[l..r]". Trampa: el arreglo `p` mide n+1.

```
vector<ll> p(n + 1, 0);
for (int i = 0; i < n; i++) p[i + 1] = p[i] + a[i];
// suma a[l..r] (0-indexado, inclusivo) = p[r + 1] - p[l]
```

[A2] Sumas prefijas 2D — Cuándo: suma de submatrices. Trampa: índices 1-indexados en `rect`.

```
vector<vector<ll>> pref2D(vector<vector<ll>>& a){ int n = a.size(), m = a[0].size();
    vector<vector<ll>> p(n + 1, vector<ll>(m + 1, 0));
    for (int i = 1; i <= n; i++) for (int j = 1; j <= m; j++) p[i][j] = a[i-1][j-1] + p[i-1]
[j] + p[i][j-1] - p[i-1][j-1];
    return p; }
ll rect(vector<vector<ll>>& p, int r1, int c1, int r2, int c2){ // inclusivo, 1-indexado
    return p[r2][c2] - p[r1-1][c2] - p[r2][c1-1] + p[r1-1][c1-1]; }
```

[A3] Búsqueda binaria sobre la respuesta — Cuándo: "el mínimo X tal que se puede..." y si X sirve, X+1 también sirve. Trampa: `ok` debe ser monótona (F...F T...T); para "el máximo X" invierte la condición.

```
template<class F> ll firstTrue(ll lo, ll hi, F ok){
    while (lo < hi){ ll mid = lo + (hi - lo) / 2; if (ok(mid)) hi = mid; else lo = mid + 1; }
    return lo; }
// uso: ll r = firstTrue(0, 1e18, [&](ll x){ return /* se puede con x? */; });
```

[A4] Dos punteros / ventana deslizante — Cuándo: subarreglo más largo/corto que cumple algo y la condición se mantiene al encoger. Trampa: requiere valores ≥ 0 (si hay negativos usa prefijos + map).

```
int longestAtMost(vector<ll>& a, ll K){ int best = 0; ll s = 0;   // suma <= K
    for (int l = 0, r = 0; r < (int)a.size(); r++){
        s += a[r];
        while (s > K) s -= a[l++];
        best = max(best, r - l + 1); }
    return best; }
```

[G] Grafos y árboles (C++)

Todo 0-indexado. Si la entrada viene 1-indexada, resta 1 al leer (`u--, v--`).

[G1] Leer un grafo — Cuándo: siempre. Trampa: no dirigido = agrega la arista en ambos sentidos.

```
int n, m; cin >> n >> m;
vector<vector<int>> adj(n);                    // sin pesos
vector<vector<pair<int,ll>>> wadj(n);          // con pesos (vecino, peso)
for (int i = 0; i < m; i++){
    int u, v; ll w; cin >> u >> v >> w; u--; v--;
    adj[u].push_back(v); adj[v].push_back(u);
    wadj[u].push_back({v, w}); wadj[v].push_back({u, w});
}
```

[G2] BFS (distancias sin peso) — Cuándo: camino más corto con aristas de costo 1. Trampa: marca `d[v]` al encolar, no al sacar.

```
vector<int> bfs(int s, vector<vector<int>>& adj){
    vector<int> d(adj.size(), -1); queue<int> q; d[s] = 0; q.push(s);
    while (!q.empty()){ int u = q.front(); q.pop();
        for (int v : adj[u]) if (d[v] == -1){ d[v] = d[u] + 1; q.push(v); } }
    return d;   // -1 = inalcanzable
}
```

[G3] BFS en grilla — Cuándo: laberintos, `#` = pared. Trampa: revisa límites ANTES de leer `g[nr][nc]`.

```
int dr[4] = {-1,1,0,0}, dc[4] = {0,0,-1,1};
vector<vector<int>> bfsGrid(vector<string>& g, int sr, int sc){
    int R = g.size(), C = g[0].size();
    vector<vector<int>> d(R, vector<int>(C, -1)); queue<pair<int,int>> q;
    d[sr][sc] = 0; q.push({sr, sc});
    while (!q.empty()){ auto [r, c] = q.front(); q.pop();
        for (int k = 0; k < 4; k++){ int nr = r + dr[k], nc = c + dc[k];
            if (nr < 0 || nr >= R || nc < 0 || nc >= C) continue;
            if (g[nr][nc] == '#' || d[nr][nc] != -1) continue;
            d[nr][nc] = d[r][c] + 1; q.push({nr, nc}); } }
    return d;
}
```

[G4] DFS iterativo / componentes conexas — Cuándo: contar componentes, alcanzabilidad. Trampa: DFS recursivo con N = 10^5 en línea puede desbordar la pila; esta versión no.

```
vector<int> components(vector<vector<int>>& adj, int &cnt){
    int n = adj.size(); vector<int> comp(n, -1); cnt = 0;
    for (int s = 0; s < n; s++) if (comp[s] == -1){
        stack<int> st; st.push(s); comp[s] = cnt;
        while (!st.empty()){ int u = st.top(); st.pop();
            for (int v : adj[u]) if (comp[v] == -1){ comp[v] = cnt; st.push(v); } }
        cnt++; }
    return comp;
}
```

[G5] Dijkstra (+ omitir un nodo) — Cuándo: caminos más cortos con pesos ≥ 0. `banned` sirve para el truco "¿todos los caminos óptimos pasan por G?": corre con y sin G y compara. Trampa: `if (du > d[u]) continue;` es obligatorio.

```
vector<ll> dijkstra(int s, vector<vector<pair<int,ll>>>& adj, int banned = -1){
    vector<ll> d(adj.size(), INF);
    priority_queue<pair<ll,int>, vector<pair<ll,int>>, greater<>> pq;
    d[s] = 0; pq.push({0, s});
    while (!pq.empty()){ auto [du, u] = pq.top(); pq.pop();
        if (du > d[u]) continue;
        for (auto [v, w] : adj[u]){ if (v == banned) continue;
            if (d[u] + w < d[v]){ d[v] = d[u] + w; pq.push({d[v], v}); } } }
    return d;
}
```

[G6] Grafo invertido + varias búsquedas — Cuándo: "¿qué nodos llegan a X?" o "¿existe un v alcanzable por p, a y b a la vez?". En vez de un BFS por nodo (O(n²) BFS), invierte aristas y haz un BFS desde cada nodo especial.

```
vector<vector<int>> radj(n);
for (int u = 0; u < n; u++) for (int v : adj[u]) radj[v].push_back(u);
auto dp = bfs(p, radj), da = bfs(a, radj), db = bfs(b, radj);
for (int v = 0; v < n; v++) if (dp[v] != -1 && da[v] != -1 && db[v] != -1) { /* v llega a p,
a y b */ }
```

[G7] Orden topológico (Kahn) + DP en DAG — Cuándo: dependencias, "camino más largo", contar caminos en un DAG. Trampa: si `order.size() < n` hay ciclo.

```
vector<int> toposort(vector<vector<int>>& adj){
    int n = adj.size(); vector<int> indeg(n, 0), order;
    for (int u = 0; u < n; u++) for (int v : adj[u]) indeg[v]++;
    queue<int> q; for (int u = 0; u < n; u++) if (!indeg[u]) q.push(u);
    while (!q.empty()){ int u = q.front(); q.pop(); order.push_back(u);
        for (int v : adj[u]) if (--indeg[v] == 0) q.push(v); }
    return order;
}
// DP: camino mas largo
vector<int> lp(n, 0);
for (int u : order) for (int v : adj[u]) lp[v] = max(lp[v], lp[u] + 1);
```

[G8] DSU (Union-Find) — Cuándo: unir grupos y preguntar "¿están juntos?", Kruskal. Trampa: compara siempre `find(a) == find(b)`, nunca `p[a] == p[b]`.

```
struct DSU {
    vector<int> p, sz;
    DSU(int n) : p(n), sz(n, 1) { iota(p.begin(), p.end(), 0); }
    int find(int x){ return p[x] == x ? x : p[x] = find(p[x]); }
    bool unite(int a, int b){
        a = find(a); b = find(b); if (a == b) return false;
        if (sz[a] < sz[b]) swap(a, b);
        p[b] = a; sz[a] += sz[b]; return true; }
};
// Kruskal: ordena aristas por peso; si dsu.unite(u, v) -> suma el peso al MST
```

[G9] Árbol DFS: back-edges, puentes, articulaciones — Cuándo: ciclos que no desconectan, aristas críticas, biconexidad. `back` guarda (descendiente, ancestro); `dep` es la profundidad. Trampa: recursivo; N ≤ ~2·10^5 suele estar bien.

```
struct DfsTree {
    int n, timer = 0; vector<vector<pair<int,int>>> adj;   // (vecino, id arista)
    vector<int> tin, low, dep, par; vector<bool> isArt; vector<int> bridges;
vector<array<int,2>> back;
    DfsTree(int n) : n(n), adj(n), tin(n, -1), low(n), dep(n), par(n, -1), isArt(n) {}
    void addEdge(int u, int v, int id){ adj[u].push_back({v, id}); adj[v].push_back({u, id});
}
    void dfs(int u, int pe){ tin[u] = low[u] = timer++; int ch = 0;
        for (auto [v, id] : adj[u]){ if (id == pe) continue;
            if (tin[v] != -1){ low[u] = min(low[u], tin[v]); if (tin[v] < tin[u])
back.push_back({u, v}); }
            else { par[v] = u; dep[v] = dep[u] + 1; dfs(v, id); low[u] = min(low[u], low[v]);
ch++;
                if (low[v] > tin[u]) bridges.push_back(id);
                if (pe != -1 && low[v] >= tin[u]) isArt[u] = true; } }
        if (pe == -1 && ch > 1) isArt[u] = true; }
    void run(){ for (int i = 0; i < n; i++) if (tin[i] == -1) dfs(i, -1); }
};
```

[G10] LCA con binary lifting — Cuándo: distancia entre nodos de un árbol, k-ésimo ancestro, saltos sobre caminos. O(N log N) preproceso, O(log N) por consulta.

```
struct LCA {
    int n, LOG; vector<vector<int>> up; vector<int> dep;
    LCA(vector<vector<int>>& adj, int root = 0) : n(adj.size()) {
        LOG = 1; while ((1 << LOG) < n) LOG++;
        up.assign(LOG, vector<int>(n, root)); dep.assign(n, 0);
        vector<int> st = {root}; vector<bool> vis(n); vis[root] = true;
        while (!st.empty()){ int u = st.back(); st.pop_back();
            for (int v : adj[u]) if (!vis[v]){ vis[v] = true; up[0][v] = u; dep[v] = dep[u] +
1; st.push_back(v); } }
        for (int k = 1; k < LOG; k++) for (int v = 0; v < n; v++) up[k][v] = up[k-1][up[k-1]
[v]];
    }
    int kth(int v, int k){ for (int i = 0; i < LOG; i++) if (k >> i & 1) v = up[i][v]; return
v; }
    int lca(int a, int b){ if (dep[a] < dep[b]) swap(a, b); a = kth(a, dep[a] - dep[b]);
        if (a == b) return a;
        for (int k = LOG - 1; k >= 0; k--) if (up[k][a] != up[k][b]){ a = up[k][a]; b = up[k]
[b]; }
        return up[0][a]; }
    int dist(int a, int b){ return dep[a] + dep[b] - 2 * dep[lca(a, b)]; }
};
```

[G11] Rerooting (DP con cambio de raíz) — Cuándo: "para CADA nodo como raíz, calcula…" en O(N). Patrón: (1) DP de hijos hacia el padre en orden inverso; (2) pasa la respuesta del padre al hijo. Ejemplo: suma de distancias de cada nodo a todos.

```
vector<ll> sumDistAll(vector<vector<int>>& adj){
    int n = adj.size(); vector<int> order, par(n, -1); vector<ll> sz(n, 1), down(n, 0),
ans(n);
    vector<int> st = {0}; par[0] = 0;
    while (!st.empty()){ int u = st.back(); st.pop_back(); order.push_back(u);
        for (int v : adj[u]) if (par[v] == -1){ par[v] = u; st.push_back(v); } }
    par[0] = -1;
    for (int i = n - 1; i >= 0; i--){ int u = order[i];        // 1) hijos -> padre
        for (int v : adj[u]) if (v != par[u]){ sz[u] += sz[v]; down[u] += down[v] + sz[v]; }
}
    ans[0] = down[0];
    for (int u : order) for (int v : adj[u]) if (v != par[u])   // 2) padre -> hijos
        ans[v] = ans[u] - sz[v] + (n - sz[v]);
    return ans;
}
```

Variante "mejor y segundo mejor": en el paso 1 guarda, por nodo, el mejor valor y el segundo mejor que venga de un hijo DISTINTO; en el paso 2, si el mejor del padre viene de v, usa el segundo.

[G12] Matching bipartito (Hopcroft-Karp) — Cuándo: emparejar máximo izquierda↔derecha (tareas/personas, filas/columnas, color de texto/color de fondo). O(E√V). Conjunto independiente máximo en bipartito = V − matching.

```
struct HopcroftKarp {
    int L, R; vector<vector<int>> adj; vector<int> mL, mR, dist;
    HopcroftKarp(int L, int R) : L(L), R(R), adj(L), mL(L, -1), mR(R, -1), dist(L) {}
    void addEdge(int u, int v){ adj[u].push_back(v); }        // u en [0,L), v en [0,R)
    bool bfs(){ queue<int> q; bool found = false;
        for (int u = 0; u < L; u++){ if (mL[u] == -1){ dist[u] = 0; q.push(u); } else dist[u]
= -1; }
        while (!q.empty()){ int u = q.front(); q.pop();
            for (int v : adj[u]){ int w = mR[v];
                if (w == -1) found = true; else if (dist[w] == -1){ dist[w] = dist[u] + 1;
q.push(w); } } }
        return found; }
    bool dfs(int u){ for (int v : adj[u]){ int w = mR[v];
            if (w == -1 || (dist[w] == dist[u] + 1 && dfs(w))){ mL[u] = v; mR[v] = u; return
true; } }
        dist[u] = -1; return false; }
    int maxMatching(){ int res = 0; while (bfs()) for (int u = 0; u < L; u++) if (mL[u] == -1
&& dfs(u)) res++; return res; }
};
```

[G13] Flujo máximo / corte mínimo (Dinic) — Cuándo: capacidades, "mínimo de aristas a cortar para separar s de t", asignaciones con límites. Truco: un nodo con capacidad c se divide en `v_in` → `v_out` con capacidad c.

```
struct Dinic {
    struct E { int to; ll cap; };
    vector<E> e; vector<vector<int>> g; vector<int> lvl, it;
    Dinic(int n) : g(n), lvl(n), it(n) {}
    void addEdge(int u, int v, ll c){ g[u].push_back(e.size()); e.push_back({v, c});
g[v].push_back(e.size()); e.push_back({u, 0}); }
    bool bfs(int s, int t){ fill(lvl.begin(), lvl.end(), -1); lvl[s] = 0; queue<int> q;
q.push(s);
        while (!q.empty()){ int u = q.front(); q.pop();
            for (int id : g[u]) if (e[id].cap > 0 && lvl[e[id].to] == -1){ lvl[e[id].to] =
lvl[u] + 1; q.push(e[id].to); } }
        return lvl[t] != -1; }
    ll dfs(int u, int t, ll f){ if (u == t) return f;
        for (int &i = it[u]; i < (int)g[u].size(); i++){ int id = g[u][i], v = e[id].to;
            if (e[id].cap > 0 && lvl[v] == lvl[u] + 1){ ll got = dfs(v, t, min(f,
e[id].cap));
                if (got){ e[id].cap -= got; e[id ^ 1].cap += got; return got; } } }
        return 0; }
    ll maxflow(int s, int t){ ll fl = 0; while (bfs(s, t)){ fill(it.begin(), it.end(), 0);
while (ll f = dfs(s, t, INF)) fl += f; } return fl; }
    // corte minimo: tras maxflow, los nodos con lvl[v] != -1 estan del lado de s
};
```

[D] Estructuras de datos (C++)

¿Cuál uso?

Necesito…	Estructura
Suma en rango + sumar en un punto	[D1] Fenwick
k-ésimo menor, "cuántos son < x" con inserciones/borrados	[D2] Ordered set (o [D1] kth)
Sumar a un rango + suma de un rango	[D3] Segment tree lazy
Mínimo/máximo en rango SIN actualizaciones	[D4] Sparse table O(1)
Máximo de rectas a·x + b (o funciones que se cruzan a lo sumo una vez)	[D5] Li-Chao
Actualizaciones que obligan a recalcular todo	[D6] Sqrt batching

[D1] Fenwick / BIT — Índices 1..n. `kth(k)` = menor posición cuya suma prefija ≥ k (valores ≥ 0). Trampa: ¡nunca uses el índice 0!

```
struct BIT {
    int n; vector<ll> t;
    BIT(int n) : n(n), t(n + 1, 0) {}
    void add(int i, ll v){ for (; i <= n; i += i & -i) t[i] += v; }
    ll sum(int i){ ll s = 0; for (; i > 0; i -= i & -i) s += t[i]; return s; }
    ll sum(int l, int r){ return sum(r) - sum(l - 1); }
    int kth(ll k){ int pos = 0;
        for (int pw = 1 << __lg(n); pw; pw >>= 1)
            if (pos + pw <= n && t[pos + pw] < k){ pos += pw; k -= t[pos]; }
        return pos + 1; }
};
// "k-esimo disponible": add(i, 1) para todos; tomar x = kth(k); add(x, -1)
```

[D2] Ordered set (PBDS) — Cuándo: un `set` que además responde "k-ésimo" y "posición de x" en O(log n). Trampa: no admite repetidos; para multiset guarda `pair<valor, id_unico>`. Pon estas líneas debajo de `using namespace std;`.

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
typedef tree<int, null_type, less<int>, rb_tree_tag, tree_order_statistics_node_update>
ordered_set;
ordered_set os; os.insert(5); os.erase(5);
*os.find_by_order(k);   // k-esimo (0-indexado)
os.order_of_key(x);     // cuantos elementos son < x
```

[D3] Segment tree con lazy (sumar en rango, suma de rango) — Índices 0..n-1, rangos inclusivos. Para mínimo: cambia `+` por `min`, el neutro 0 por INF, y `t[x] += v*(largo)` por `t[x] += v`.

```
struct SegLazy {
    int n; vector<ll> t, lz;
    SegLazy(int n) : n(n), t(4 * n, 0), lz(4 * n, 0) {}
    void push(int x, int l, int r){ if (!lz[x]) return; int m = (l + r) / 2;
        t[2*x] += lz[x] * (m - l + 1); lz[2*x] += lz[x];
        t[2*x+1] += lz[x] * (r - m); lz[2*x+1] += lz[x]; lz[x] = 0; }
    void update(int a, int b, ll v, int x, int l, int r){
        if (b < l || r < a) return;
        if (a <= l && r <= b){ t[x] += v * (r - l + 1); lz[x] += v; return; }
        push(x, l, r); int m = (l + r) / 2;
        update(a, b, v, 2*x, l, m); update(a, b, v, 2*x+1, m + 1, r); t[x] = t[2*x] +
t[2*x+1]; }
    ll query(int a, int b, int x, int l, int r){
        if (b < l || r < a) return 0;
        if (a <= l && r <= b) return t[x];
        push(x, l, r); int m = (l + r) / 2;
        return query(a, b, 2*x, l, m) + query(a, b, 2*x+1, m + 1, r); }
    void update(int a, int b, ll v){ update(a, b, v, 1, 0, n - 1); }
    ll query(int a, int b){ return query(a, b, 1, 0, n - 1); }
};
```

[D4] Sparse table (RMQ en O(1)) — Cuándo: arreglo fijo, miles de consultas min/max. Sirve para min, max, gcd (operaciones idempotentes), no para suma.

```
struct Sparse {
    vector<vector<ll>> st;
    Sparse(vector<ll>& a){ int n = a.size(), K = __lg(n) + 1; st.assign(K, a);
        for (int k = 1; k < K; k++) for (int i = 0; i + (1 << k) <= n; i++)
            st[k][i] = min(st[k-1][i], st[k-1][i + (1 << (k-1))]); }
    ll query(int l, int r){ int k = __lg(r - l + 1); return min(st[k][l], st[k][r - (1 << k)
+ 1]); }
};
```

[D5] Li-Chao tree (máximo de rectas) — Cuándo: insertar rectas a·x+b y preguntar el máximo en un x entero de [lo, hi]; también con cualquier familia donde dos funciones se cruzan a lo sumo una vez (cambia `eval`). Para mínimo: inserta (−a, −b) y niega la respuesta. Trampa: cuida que a·x no desborde `ll`.

```
struct LiChao {
    struct Line { ll a, b; ll eval(ll x) const { return a * x + b; } };
    struct Node { Line ln; int l = -1, r = -1; };
    vector<Node> t; ll lo, hi;
    LiChao(ll lo, ll hi) : lo(lo), hi(hi) { t.push_back({{0, LLONG_MIN / 2}}); }
    void add(Line nw, int x, ll l, ll r){
        ll m = l + (r - l) / 2;
        bool lef = nw.eval(l) > t[x].ln.eval(l), mid = nw.eval(m) > t[x].ln.eval(m);
        if (mid) swap(t[x].ln, nw);
        if (l == r) return;
        if (lef != mid){ if (t[x].l == -1){ t[x].l = t.size(); t.push_back({nw}); } else
add(nw, t[x].l, l, m); }
        else { if (t[x].r == -1){ t[x].r = t.size(); t.push_back({nw}); } else add(nw,
t[x].r, m + 1, r); } }
    ll query(ll q, int x, ll l, ll r){ if (x == -1) return LLONG_MIN / 2;
        ll m = l + (r - l) / 2, res = t[x].ln.eval(q);
        if (l == r) return res;
        return max(res, q <= m ? query(q, t[x].l, l, m) : query(q, t[x].r, m + 1, r)); }
    void add(ll a, ll b){ add({a, b}, 0, lo, hi); }
    ll query(ll x){ return query(x, 0, lo, hi); }
};
// uso: LiChao lc(-1e9, 1e9); lc.add(a, b); lc.query(x);
```

[D6] Sqrt batching (esqueleto) — Cuándo: hay un recalculo global caro (O(N) o O(M), p. ej. DP en orden topológico) y las actualizaciones lo invalidan. Agrupa las actualizaciones en bloques de tamaño B ≈ √Q: entre recalculos, responde con el valor base + corrección por las ≤ B actualizaciones pendientes.

```
const int B = 400;                      // ~ sqrt(numero de operaciones)
vector<Upd> pend;                       // actualizaciones aun no aplicadas al "base"
rebuild();                              // calcula base[] con todo aplicado, O(N)
for (auto &op : ops){
    if (op.esUpdate){
        pend.push_back(op.upd);
        if ((int)pend.size() == B){ aplicar(pend); rebuild(); pend.clear(); }
    } else {
        ll r = base[op.i];
        for (auto &u : pend) r += efecto(u, op.i);   // correccion O(B)
        cout << r << "\n";
    }
}
// total: O((Q/B)*N + Q*B), minimo con B ~ sqrt(N)
```

[S] Strings avanzados

¿Cuál uso?

Necesito…	Técnica
Comparar dos subcadenas en O(1), o hashear un estado (vector de frecuencias)	[S1] Rolling hash
Todas las apariciones de un patrón	[S2] KMP
Prefijo común más largo entre s y cada sufijo	[S3] Z-function
Contar subcadenas con paréntesis balanceados	[S4]
Validar expresiones / gramática	[S5] Reducción canónica
N ≤ 10^4 y solo manipulación	Python [P3]

[S1] Rolling hash (mod 2^61−1, base aleatoria) — `get(l, r)` = hash de s[l..r) con r EXCLUSIVO. Base aleatoria = resistente a casos anti-hash. Trampa: si dos hashes coinciden, las subcadenas son iguales con probabilidad altísima, no 100 %.

```
typedef unsigned long long ull;
const ull MOD = (1ULL << 61) - 1;
ull mulmod(ull a, ull b){ __uint128_t c = (__uint128_t)a * b; ull r = (ull)(c & MOD) + (ull)
(c >> 61); return r >= MOD ? r - MOD : r; }
mt19937_64 rng64(chrono::steady_clock::now().time_since_epoch().count());
const ull BASE = rng64() % (MOD - 1000) + 500;
struct Hash {
    vector<ull> h, pw;
    Hash(const string& s) : h(s.size() + 1, 0), pw(s.size() + 1, 1) {
        for (size_t i = 0; i < s.size(); i++){
            h[i+1] = (mulmod(h[i], BASE) + (ull)s[i]) % MOD; pw[i+1] = mulmod(pw[i], BASE); }
}
    ull get(int l, int r){ return (h[r] + MOD - mulmod(h[l], pw[r - l])) % MOD; }
};
```

Hash de un vector de frecuencias que cambia: guarda H = Σ freq[i]·BASE^i; al hacer `freq[i]++` suma BASE^i a H (O(1)). Restar 1 a todas las entradas = restar Σ BASE^i (precálculalo). Guarda cada H en un `map<ull,int>` con su primer índice.

[S2] KMP: función prefijo + buscar patrón — O(|txt| + |pat|). `pi[i]` = largo del borde más largo de s[0..i]. Trampa: el separador # no debe aparecer en el texto.

```
vector<int> prefixFn(const string& s){ int n = s.size(); vector<int> pi(n, 0);
    for (int i = 1; i < n; i++){ int j = pi[i-1]; while (j > 0 && s[i] != s[j]) j = pi[j-1];
if (s[i] == s[j]) j++; pi[i] = j; }
    return pi; }
vector<int> findAll(const string& txt, const string& pat){   // posiciones de inicio
    string s = pat + "#" + txt; auto pi = prefixFn(s); vector<int> res; int m = pat.size();
    for (int i = m + 1; i < (int)s.size(); i++) if (pi[i] == m) res.push_back(i - 2 * m);
    return res; }
```

[S3] Z-function — `z[i]` = largo del mayor prefijo común entre s y s[i..]. `z[0]` queda en 0.

```
vector<int> zfn(const string& s){ int n = s.size(); vector<int> z(n, 0);
    for (int i = 1, l = 0, r = 0; i < n; i++){ if (i < r) z[i] = min(r - i, z[i - l]);
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++; if (i + z[i] > r){ l = i; r =
i + z[i]; } }
    return z; }
```

[S4] Contar subcadenas con paréntesis balanceados — O(n log n). Idea: [l, r) es válida si el balance prefijo en l y r es igual y nunca baja de ese nivel entre medio; al bajar de nivel se "olvidan" los inicios de ese nivel.

```
ll countBalanced(const string& s){   // '(' = +1, ')' = -1
    // c[b] = # prefijos con balance b que aun pueden iniciar una subcadena valida
    ll res = 0; int bal = 0; map<int,ll> c; c[0] = 1;
    for (char ch : s){
        if (ch == '('){ bal++; c[bal]++; }
        else { c[bal] = 0; bal--; res += c[bal]; c[bal]++; } }
    return res; }
```

[S5] Reducción canónica (parsing sin gramática) — Cuándo: validar o contar expresiones. Pasos: (1) reemplaza cada nombre de variable por `a` y cada operador binario por `+`; (2) la expresión es válida si los paréntesis están balanceados, no empieza ni termina con `+` y no contiene ninguno de los pares prohibidos `()` `(+` `+)` `++` `)(` `)a` `a(`; (3) corta la cadena en los pares prohibidos y cuenta en cada trozo con [S4] adaptado.

```
import re
def canon(s):                                   # Python: ideal para este paso
    s = re.sub(r"[A-Za-z_][A-Za-z_0-9]*", "a", s)
    return re.sub(r"[-*/+]", "+", s)
PROHIBIDOS = ["()", "(+", "+)", "++", ")(", ")a", "a("]
```

[M] Matemática, combinatoria y juegos

[M1] Aritmética modular y combinatoria — Cuándo: "imprime la respuesta módulo 10^9+7". Trampa: reduce con `% MD` después de CADA producto; `inv` solo sirve si MD es primo. Llama `initFact()` una vez en `main`.

```
const ll MD = 1e9 + 7;
ll pw(ll b, ll e, ll m = MD){ ll r = 1; b %= m; if (b < 0) b += m;
    while (e){ if (e & 1) r = r * b % m; b = b * b % m; e >>= 1; } return r; }
ll inv(ll a){ return pw(a, MD - 2); }
const int MAXF = 1000005; ll fact[MAXF], ifact[MAXF];
void initFact(){ fact[0] = 1; for (int i = 1; i < MAXF; i++) fact[i] = fact[i-1] * i % MD;
    ifact[MAXF-1] = inv(fact[MAXF-1]); for (int i = MAXF - 1; i > 0; i--) ifact[i-1] =
ifact[i] * i % MD; }
ll C(int n, int k){ if (k < 0 || k > n) return 0; return fact[n] * ifact[k] % MD * ifact[n-k]
% MD; }
```

Fórmulas útiles: estrellas y barras (repartir n iguales en k cajas) = C(n+k−1, k−1); caminos en grilla a×b = C(a+b, a); Catalan(n) = C(2n, n)/(n+1).

[M2] Criba, divisores, factorización — `spf[x]` = menor primo que divide x (factoriza cualquier x ≤ n en O(log x)). `divisors` y `factorize` son O(√n).

```
vector<int> sieve(int n){ vector<int> spf(n + 1, 0);
    for (int i = 2; i <= n; i++) if (!spf[i]) for (int j = i; j <= n; j += i) if (!spf[j])
spf[j] = i;
    return spf; }   // x es primo <=> spf[x] == x
vector<ll> divisors(ll n){ vector<ll> d;
    for (ll i = 1; i * i <= n; i++) if (n % i == 0){ d.push_back(i); if (i != n / i)
d.push_back(n / i); }
    sort(d.begin(), d.end()); return d; }
vector<pair<ll,int>> factorize(ll n){ vector<pair<ll,int>> f;
    for (ll p = 2; p * p <= n; p++) if (n % p == 0){ int e = 0; while (n % p == 0){ n /= p;
e++; } f.push_back({p, e}); }
    if (n > 1) f.push_back({n, 1}); return f; }
```

[M3] Bases numéricas y palíndromos — Cuándo: "¿en qué bases b es N palíndromo?". Truco O(√N): para b ≤ √N prueba con fuerza bruta; para b > √N el número tiene 2 dígitos `dd` y N = d·(b+1), así que b = N/d − 1 para cada divisor d de N con d < b.

```
vector<ll> toBase(ll n, ll b){ vector<ll> d; if (n == 0) d.push_back(0);
    while (n){ d.push_back(n % b); n /= b; } reverse(d.begin(), d.end()); return d; }
bool isPal(const vector<ll>& d){ for (size_t i = 0, j = d.size() - 1; i < j; i++, j--) if
(d[i] != d[j]) return false; return true; }
```

[M4] Permutaciones con x inversiones (DP + sumas prefijas) — f[n][x] = Σ f[n−1][x−j] para 0 ≤ j < n, optimizado a O(n·X). `cap` evita desborde cuando K llega a 10^18.

```
vector<vector<ll>> invCount(int N, int X, ll cap = LLONG_MAX / 4){
    vector<vector<ll>> f(N + 1, vector<ll>(X + 1, 0)); f[0][0] = 1;
    for (int n = 1; n <= N; n++){ vector<ll> g(X + 1);          // g = prefijo de f[n-1]
        for (int x = 0; x <= X; x++) g[x] = min(cap, (x ? g[x-1] : 0) + f[n-1][x]);
        for (int x = 0; x <= X; x++) f[n][x] = min(cap, g[x] - (x - n >= 0 ? g[x-n] : 0)); }
    return f; }
```

K-ésima permutación con x inversiones: con r elementos restantes, elegir el j-ésimo menor disponible (j = 0..r−1) aporta j inversiones y deja f[r−1][x−j] completaciones. Recorre j: si K > ese número, réstalo de K; si no, toma ese elemento (extráelo con [D1] `kth` o [D2]) y resta j a x.

[M5] Juegos: Grundy y mex — Juego imparcial: grundy(estado) = mex de los grundy de los estados alcanzables; varios juegos independientes → XOR de sus grundy; el primero pierde si el XOR es 0. Antes de programar Grundy, prueba casos pequeños a mano y busca un invariante (paridad, módulo 3, simetría/imitar al rival).

```
int mex(vector<int>& v){ set<int> s(v.begin(), v.end()); int m = 0; while (s.count(m)) m++;
return m; }
```

[M6] Patrones constructivos — Cuándo: "construye una secuencia/arreglo que cumpla...".

Patrón	Idea
Pensar hacia atrás	Simula desde el estado final al inicial (pliegues, eliminaciones)
Extremos + inducción	Demuestra dónde va el 1 (o el máximo), quítalo y repite con n−1; dos punteros `l`, `r`
Sistema lineal de bloques	Si la solución se arma con bloques de tamaño 2 y 3: n = 2·a + 3·b y k = a + b → a = 3k − n, b = n − 2k (verifica a, b ≥ 0)
Fuerza bruta pequeña	Genera todas las respuestas para n ≤ 8 en Python y busca el patrón

[GE] Geometría en enteros (C++)

Regla: si las coordenadas son enteras, trabaja TODO en `ll` (producto cruz, punto, shoelace). Coordenadas ≤ 10^9 → un cruz cabe en `ll` (≤ ~4·10^18).

[GE1] Punto, cruz, orientación, área — `orient(a,b,c)` > 0 si c está a la IZQUIERDA de a→b (giro antihorario), < 0 a la derecha, 0 colineal. `area2` devuelve 2·área con signo (positiva si el polígono va antihorario).

```
struct P { ll x, y;
    P operator-(const P& o) const { return {x - o.x, y - o.y}; }
    P operator+(const P& o) const { return {x + o.x, y + o.y}; }
    bool operator<(const P& o) const { return x != o.x ? x < o.x : y < o.y; }
    bool operator==(const P& o) const { return x == o.x && y == o.y; } };
ll cross(P a, P b){ return a.x * b.y - a.y * b.x; }
ll dot(P a, P b){ return a.x * b.x + a.y * b.y; }
ll orient(P a, P b, P c){ return cross(b - a, c - a); }
ll area2(vector<P>& p){ ll s = 0; int n = p.size();
    for (int i = 0; i < n; i++) s += cross(p[i], p[(i + 1) % n]); return s; }
```

[GE2] Envolvente convexa (monotone chain) — O(n log n), antihorario, sin puntos colineales. No la modifiques para incluir colineales: cambiar a `< 0` falla si todos los puntos están en una recta.

```
vector<P> hull(vector<P> p){ sort(p.begin(), p.end()); p.erase(unique(p.begin(), p.end()),
p.end());
    int n = p.size(); if (n < 3) return p; vector<P> h(2 * n); int k = 0;
    for (int i = 0; i < n; i++){ while (k >= 2 && orient(h[k-2], h[k-1], p[i]) <= 0) k--;
h[k++] = p[i]; }
    for (int i = n - 2, t = k + 1; i >= 0; i--){ while (k >= t && orient(h[k-2], h[k-1],
p[i]) <= 0) k--; h[k++] = p[i]; }
    h.resize(k - 1); return h; }
```

[GE3] ¿Se cortan dos segmentos? — Incluye tocarse en un extremo y solaparse colineales.

```
bool onSeg(P a, P b, P p){ return orient(a, b, p) == 0 && min(a.x,b.x) <= p.x && p.x <=
max(a.x,b.x)
                               && min(a.y,b.y) <= p.y && p.y <= max(a.y,b.y); }
int sgn(ll v){ return (v > 0) - (v < 0); }
bool segInter(P a, P b, P c, P d){
    int o1 = sgn(orient(a,b,c)), o2 = sgn(orient(a,b,d)), o3 = sgn(orient(c,d,a)), o4 =
sgn(orient(c,d,b));
    if (o1 != o2 && o3 != o4) return true;
    return onSeg(a,b,c) || onSeg(a,b,d) || onSeg(c,d,a) || onSeg(c,d,b); }
```

[GE4] Orden angular (barrido radial) — Ordena vectores por ángulo desde 0° hasta 360°, sin `atan2` ni flotantes. Para ordenar alrededor de un pivote q, ordena `p[i] - q`.

```
int half(P p){ return p.y < 0 || (p.y == 0 && p.x < 0); }   // 0: [0,180)  1: [180,360)
bool angLess(P a, P b){ if (half(a) != half(b)) return half(a) < half(b); return cross(a, b)
> 0; }
// sort(v.begin(), v.end(), angLess);
```

Otras técnicas que aparecen en el regional (sin plantilla aquí): rotating calipers sobre el hull (diámetro, ancho), línea de barrido con eventos ordenados, intersección de semiplanos O(n log n).

[T] Esqueletos de problema completos

Copia el que se parezca a la entrada, luego rellena el centro con la plantilla de la sección que toque.

[T1] Python — "N y una lista" (con o sin varios casos)

```
import sys
input = sys.stdin.readline

def main():
    t = int(input())                # quita estas 2 lineas si hay un solo caso
    for _ in range(t):
        n = int(input())
        a = list(map(int, input().split()))
        print(sum(a))

main()
```

[T2] Python — leer todo de golpe (entrada grande o formato raro)

```
import sys
data = sys.stdin.buffer.read().split()   # todo el input de golpe: lo mas rapido
pos = 0
def nxt():
    global pos
    pos += 1
    return data[pos - 1]
n, m = int(nxt()), int(nxt())
g = [[] for _ in range(n)]
for _ in range(m):
    u, v = int(nxt()) - 1, int(nxt()) - 1
    g[u].append(v); g[v].append(u)
```

Trampa: `data` contiene bytes; para strings usa `nxt().decode()`.

[T3] C++ — "grafo con N nodos y M aristas" (BFS desde el nodo 1)

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
int main(){
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int n, m; cin >> n >> m;
    vector<vector<int>> adj(n);
    for (int i = 0; i < m; i++){ int u, v; cin >> u >> v; u--; v--; adj[u].push_back(v);
adj[v].push_back(u); }
    vector<int> d(n, -1); queue<int> q; d[0] = 0; q.push(0);
    while (!q.empty()){ int u = q.front(); q.pop();
        for (int v : adj[u]) if (d[v] == -1){ d[v] = d[u] + 1; q.push(v); } }
    for (int i = 0; i < n; i++) cout << d[i] << " \n"[i == n - 1];
}
```

[T4] C++ — varios casos de prueba — Trampa: reinicia TODAS las estructuras globales dentro del bucle.

```
int main(){
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int t; cin >> t;
    while (t--){
        int n; cin >> n;
        vector<ll> a(n); for (auto &x : a) cin >> x;
        cout << accumulate(a.begin(), a.end(), 0LL) << "\n";
    }
}
```

[T5] Interactivo (búsqueda binaria contra el juez) — Trampa: vacía el búfer tras CADA mensaje (`endl` o `cout.flush()` en C++, `print(..., flush=True)` en Python); no uses `sync_with_stdio(false)` sin hacer flush.

```
int main(){
    int lo = 1, hi = 1000000;
    while (lo < hi){
        int mid = lo + (hi - lo) / 2;
        cout << "? " << mid << endl;        // endl SI aqui: hace flush
        string r; cin >> r;                 // formato de respuesta segun el enunciado
        if (r == "<") hi = mid; else lo = mid + 1;
    }
    cout << "! " << lo << endl;
}
```

[T6] Probar contra fuerza bruta (cuando da WA y no ves por qué) — Escribe `brute.py` lento pero obvio, un generador de casos pequeños, y compara.

```
for i in $(seq 1 500); do
  python3 gen.py $i > in.txt
  ./A < in.txt > o1.txt; python3 brute.py < in.txt > o2.txt
  if ! cmp -s o1.txt o2.txt; then echo "FALLA"; cat in.txt; break; fi
done
```

[R] Mapa: problema regional → técnica → plantilla

[R1] Señales en el enunciado

Si el enunciado dice…	Piensa en…	Ir a
"mínimo/máximo valor tal que se pueda"	Búsqueda binaria sobre la respuesta	[A3]
"subarreglo/subcadena más larga que cumple"	Dos punteros o prefijos + map	[A4] [A1]
"camino más corto", pesos 1	BFS	[G2] [G3]
"camino más corto", pesos variables ≥ 0	Dijkstra	[G5]
"todos los caminos óptimos pasan por X"	Dijkstra con y sin X	[G5]
"qué nodos pueden llegar a X"	Invertir aristas + BFS desde X	[G6]
"tareas con dependencias", sin ciclos	Topo + DP en DAG	[G7]
"unir grupos", "¿están conectados?"	DSU	[G8]
"quitar aristas sin desconectar", "ciclo"	Árbol DFS, back-edges, puentes	[G9]
Árbol + "distancia entre u y v" / "ancestro"	LCA	[G10]
Árbol + "para cada nodo como raíz"	Rerooting	[G11]
"emparejar", "asignar sin repetir"	Matching bipartito	[G12]
"capacidad", "cortar mínimo"	Flujo / corte mínimo	[G13]
Muchas consultas de suma/min en rango	Prefijos, Fenwick, segment tree, sparse	[A1] [D1] [D3] [D4]
"k-ésimo disponible/menor"	Fenwick `kth` u ordered set	[D1] [D2]
max de $A_i \cdot X + B_i$	Li-Chao / CHT	[D5]
Actualizaciones que fuerzan recalcular todo	Sqrt batching	[D6]
"igual frecuencia", comparar subcadenas	Rolling hash	[S1]
"expresión válida", paréntesis	Reducción canónica + balance	[S4] [S5]
"módulo 10^9+7"	Aritmética modular	[M1]
N hasta 10^12 y divisores/bases	O(√N)	[M2] [M3]
"k-ésima permutación con…"	DP de conteo + reconstrucción	[M4]
"dos jugadores, juego óptimo"	Invariante o Grundy	[M5]
"construye", "rellena los ceros"	Pensar hacia atrás, extremos, bloques	[M6]
Puntos y polígonos	Cruz, hull, ángulos	[GE]

[R2] Problemas del Regional Latinoamericano por tema

Año	Problema	Técnica	Complejidad	Plantillas
2022-23	A. Asking for Money	Inversión de aristas + 3 BFS/DFS	O(N^2)	[G6]
2023	E. Elevated Profits	Eliminación inversa de hojas + buckets por promedio	O(N log N)	[M6] [C4]
2023	M. Meeting Point	Dijkstra dos veces, omitiendo un nodo	O(M log N)	[G5]
2024	B. Biketopia's Cyclic Track	Árbol DFS + back-edge de costo máximo	O(N + M)	[G9]
2024	D. Diverse T-Shirts	Cliques maximales en grafo línea + matching bipartito	O(V·E)	[G12]
2024	G. Grand Glory Race	Rerooting (mejor y segundo mejor) + LCA y saltos	O(N log N + Q $\log^2$ N)	[G11] [G10]
2024	H. Heraclosures	DP en DAG por orden topológico + sqrt batching	O(NM + E√M)	[G7] [D6]
2022-23	C. City Folding	Simulación inversa de pliegues	O(N)	[M6]
2023	B. Blackboard Game	Invariante: frecuencias módulo 3	O(N)	[M5]
2023	F. Forward and Backward	Divisores y bases palindrómicas	O(√N)	[M2] [M3]
2024	E. Evereth Expedition	Dos punteros en permutación bitónica	O(N)	[M6]
2024	F. Finding Privacy	Sistema diofántico lineal + bloques greedy	O(N)	[M6]
2024	I. Inversion Insight	DP con sumas prefijas + order statistics	O(N·X)	[M4] [D1] [D2]
2023	C. Candy Rush	Rolling hash dinámico de frecuencias	O(N log N)	[S1]
2023	D. Deciphering WordWhiz	Simulación directa sobre diccionario	O(N·Q)	[P3]
2023	L. Latam++	Reducción canónica + paréntesis	O(N)	[S5] [S4]
2024	A. Append and Panic!	Conteo con set / barrido inverso	O(|S|)	[P5]
2022-23	B. Board Game	Segment tree + CHT / Li-Chao	O(N $\log^2$ N)	[D3] [D5]
2023	A. Analyzing Contracts	Li-Chao (cruce único) o sqrt + D&C	O(N + Q log N)	[D5] [D6]
2024	C. Cindy's Christmas Challenge	Búsqueda binaria + 16 RMQ	O(W log |S|)	[A3] [D4]

Practica estos en el gimnasio de Codeforces (Latin America Regional 2020–2025) en simulacros de 5 horas.

[J] Checklist del juez BOCA y estrategia

[J1] Antes de CADA envío (cada rechazo cuesta 20 minutos de penalización)

1. El archivo se llama como la letra del problema: `A.cpp`, `A.java`, `A.py3`. En Java/Kotlin: sin `package` y la clase pública se llama `A`.
2. Sin `cerr`/prints de depuración: stderr cuenta en el límite de salida (~1 MB) y puede dar Runtime Error.
3. Formato exacto: un espacio entre valores, `\n` al final de cada línea, sin líneas vacías extra (WA vs Presentation Error).
4. `ll` donde haya sumas, productos o distancias. ¿El peor caso desborda `int`?
5. Probaste: ejemplos del enunciado + N mínimo (1) + N máximo + todos iguales + negativos/ceros si se permiten.
6. Complejidad vs límites de la tabla del índice. Si es Python con N ≥ 10^5: pásalo a C++.
7. Decimales con la cantidad de cifras pedida (`setprecision`, `:.Nf`).
8. Interactivo: flush después de cada línea.

[J2] Veredicto → qué revisar

Veredicto	Revisa primero
Wrong Answer	Desborde, caso N = 1, empates, índices 0/1, inicialización entre casos
Time Limit	Complejidad, Python → C++, `endl`, I/O rápido, bucle infinito
Runtime Error	Índice fuera de rango, división por 0, recursión profunda, arreglo enorme local, salida en stderr
Presentation Error	Espacios al final, líneas vacías, formato de decimales
Name/Problem mismatch	Nombre del archivo o de la clase

[J3] Estrategia de 5 horas (meta Top 10 SA-Norte = 4–5 problemas)

Tiempo	Qué hacer
0:00–0:20	Repartir los 12–13 enunciados entre los 3; cada uno marca fácil / medio / difícil
Todo el concurso	Mira el scoreboard: los problemas con más YES en el continente son los fáciles
0:20–1:30	Asegurar 2–3 fáciles (ad-hoc, simulación, strings, greedy). Python si cabe
1:30–4:00	Bloque medio: grafos [G], constructivos [M6], DP, estructuras [D]
4:00–5:00	Si el marcador se congela, sigue con el problema más avanzado; no abras uno nuevo en los últimos 30 min sin idea clara

Regla de equipo: una persona en el teclado, las otras dos piensan en papel; si llevas 30 min atascado en un problema, cambia de problema o de persona.